02. Enmascaramiento

Alumno: Cesar Alberto Lopez Zavala. Fecha: 18-10-2026.

Preguntas y respuestas

¿Qué información continúa existiendo en la tabla original?

Toda la información original e inalterada (los correos, teléfonos y RFCs completos).

El mecanismo implementado fue un enmascaramiento dinámico mediante una vista (VIEW).

Este enfoque intercepta la consulta y transforma el resultado en tiempo de ejecución, pero nunca modifica, trunca ni borra los datos físicos almacenados en la tabla clientes del disco duro.

¿Qué información puede visualizar el usuario soporte?

Puede visualizar el id y el nombre completos, pero los campos sensibles están parcialmente ofuscados: del correo electrónico solo ve los primeros dos caracteres y el dominio; del teléfono solo los últimos 4 dígitos; y del RFC solo los últimos 4 caracteres. El resto de los campos de la tabla original (edad, ciudad, codigo_postal, fecha_alta) no están incluidos en la vista, por lo que son completamente invisibles para este rol.

¿Por qué no es suficiente ocultar los datos únicamente en la aplicación?

Por el principio de defensa en profundidad. Si el enmascaramiento se delega exclusivamente a la aplicación web o móvil (frontend), los datos viajan completos desde la base de datos hasta la capa de aplicación. Si un usuario de soporte se conecta a la base de datos mediante otras herramientas (como psql, DBeaver, Excel o scripts), o si un atacante explota una vulnerabilidad en la API, tendrán acceso a los datos originales. Aplicar el control en el motor de base de datos asegura que la restricción sea obligatoria e inevitable, sin importar qué cliente intente acceder.

¿Qué ocurriría si soporte tuviera acceso directo a clientes?

El mecanismo de protección sería completamente inútil. El usuario podría ignorar la vista v_clientes_soporte y ejecutar un SELECT * FROM clientes; para ver toda la información en claro. Esto violaría el principio de mínimo privilegio y aumentaría innecesariamente el riesgo de exposición, extracción o divulgación no autorizada de datos personales. Por ello, la vista siempre debe ir acompañada del comando REVOKE sobre la tabla base.}

Evidencia de ejecución

Captura de la creación, carga y consulta inicial de `clientes`. Se muestran cinco filas.

Query | Query History

```
18      CONCAT(
19          '*********',
20          RIGHT(rfc, 4)
21      ) AS rfc
22  FROM clientes;
23
24  GRANT USAGE ON SCHEMA public TO soporte;
25  GRANT SELECT ON v_clientes_soporte TO soporte;
26
27  REVOKE ALL ON clientes FROM soporte;
28
29  SET ROLE soporte;
30
31  SELECT * FROM v_clientes_soporte;
32
33  SELECT * FROM clientes;
34
```

Data Output | Messages | Notifications

SQL

	id bigint	nombre character varying (120)	correo text	telefono text	rfc text
1	1	Dana Armenta	da*****@example.t...	******45...	*********5ABC
2	2	Ana Armenta	an*****@example.t...	******56...	*********0DEF
3	3	Gustavo Lopez	gu*****@example.t...	******67...	*********2GHI
4	4	Nadezhda Beltran	na*****@example.t...	******78...	*********1JKL
5	5	Adramitti Cazarez	ad*****@example.t...	******89...	*********5M...

Query | Query History

```
        '*********',
        RIGHT(rfc, 4)
    ) AS rfc
FROM clientes;

GRANT USAGE ON SCHEMA public TO soporte;
GRANT SELECT ON v_clientes_soporte TO soporte;

REVOKE ALL ON clientes FROM soporte;

SET ROLE soporte;

SELECT * FROM v_clientes_soporte;

SELECT * FROM clientes;

RESET ROLE;
```

Data Output | Messages | Notifications

```
ERROR:  permiso denegado a la tabla clientes

SQL state: 42501
```